

Neural Networks and Function Approximation

Multi-Layer Perceptrons in Scientific Machine Learning

Krishna Kumar

University of Texas at Austin

krishnak@utexas.edu

Outline

- 1 Part I: Motivation and Problem Setup
- 2 Part II: Building Neural Networks
- 3 Part III: Approximation and Expressivity
- 4 Part IV: Training Neural Networks
- 5 Part V: Architecture Design
- 6 Part VI: Practical Considerations and Summary

A displacement from noisy readings

Here is a taut string, observed at a few positions x_i with noise,

$$y_i = u(x_i) + \varepsilon_i.$$

We want the whole displacement $u(x)$, including the stretches between the readings. A coordinate network $\hat{u}(x; \theta)$ takes a position x and returns a displacement.

The parameters θ decide which function the network returns. How should we choose them?

The Poisson equation for the string

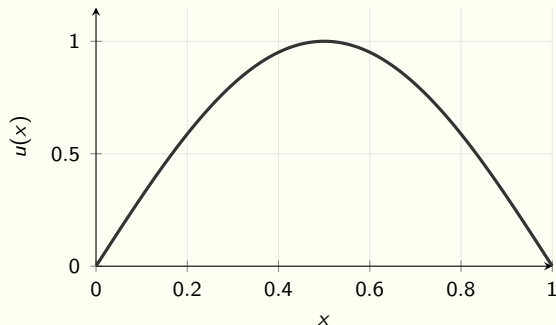
For unit tension and small slopes, the pinned string satisfies

$$-\frac{d^2 u}{dx^2} = f(x), \quad u(0) = u(1) = 0.$$

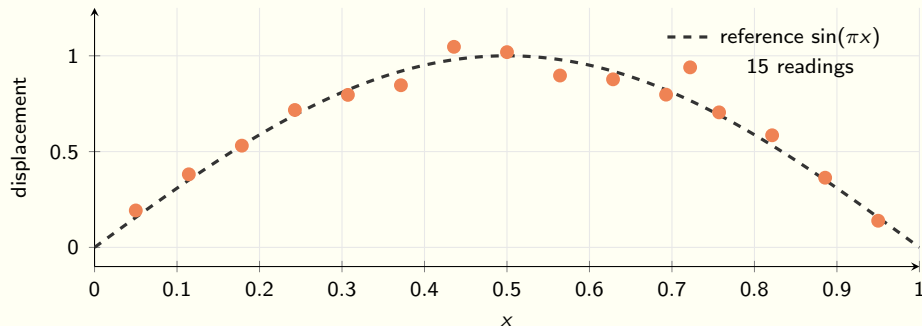
With the load $f(x) = \pi^2 \sin(\pi x)$, the displacement is

$$u(x) = \sin(\pi x).$$

This known solution lets us check every approximation in the lecture.



Fifteen noisy readings



The readings carry noise with standard deviation 0.04. In a real fitting problem the dashed reference is unknown. What decides the curve between the readings?

The data loss

The data are positions and measured displacements,

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n.$$

A network proposes a candidate function $\hat{u}(x; \theta)$. Its architecture decides which candidates exist.

The data loss compares predicted and measured values,

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{n} \sum_{i=1}^n (\hat{u}(x_i; \theta) - y_i)^2.$$

Readings held out of the fit test the curve between the readings. A small training loss alone says nothing about the curve there.

Finite differences for the Poisson equation

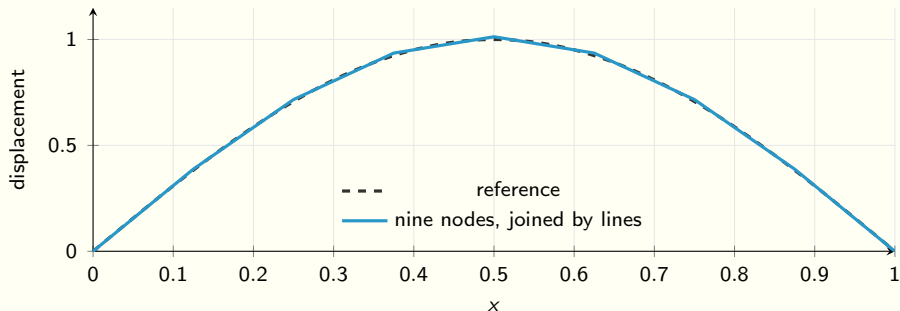
Finite differences take a different route and use the equation itself. At grid spacing h , each interior node gives

$$-\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} = f(x_i).$$

With $u_0 = u_N = 0$, these relations form a linear system for the nodal displacements.

Finite differences need the equation, the load, and the boundary values. Data fitting needs measured displacements instead.

The finite-difference solution between nodes



Nine nodal values, joined by straight lines, give a continuous curve with root-mean-square error 0.0041. The solve uses the exact load and no measurements. A network is another rule that builds a continuous curve from a few numbers.

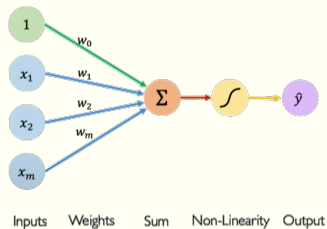
A single neuron

A neuron forms an affine combination of its inputs and applies an activation σ ,

$$z = \mathbf{w}^T \mathbf{x} + b, \quad h = \sigma(z).$$

The weights scale the inputs, and the bias shifts the sum. For the string, the input is the single coordinate x .

For classification with $\mathbf{w} \neq 0$, a sigmoid thresholded at $1/2$ splits the inputs along the hyperplane $\mathbf{w}^T \mathbf{x} + b = 0$.

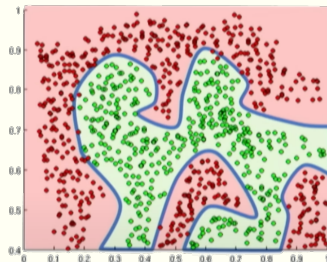
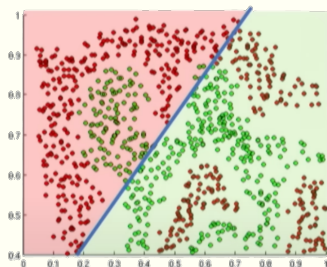


The diagram shows the mathematical formula for the neuron's output:
$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$
 The components are labeled with arrows: a purple arrow points to $\hat{y}$ with the label 'Output'; a red arrow points to w_0 with the label 'Bias'; a green arrow points to the summation term $\sum_{i=1}^m x_i w_i$ with the label 'Linear combination of inputs'; and a yellow arrow points to the function g with the label 'Non-linear activation function'.

Linear and nonlinear decision boundaries

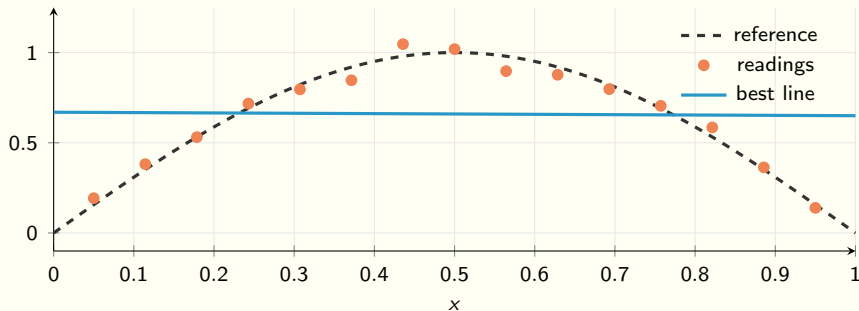
A threshold on an affine score splits the plane along a straight line,

$$\hat{y}(\mathbf{x}) = \mathbf{1}\{\mathbf{w}^T \mathbf{x} + b > 0\}.$$



On the left, the straight boundary leaves points of both classes on each side. On the right, a curved boundary separates them. Regression meets the same limit, since an affine function cannot follow a curved displacement.

An affine fit to the sine



The best line through the fifteen readings stays near 0.66, with root-mean-square error 0.31 against the sine. An affine function has one slope, and the sine needs slopes from π down to $-\pi$. Would more affine layers help?

Composition of affine layers

Without activations, two layers evaluate

$$h_1 = W_1 x + b_1,$$

$$h_2 = W_2 h_1 + b_2 = (W_2 W_1) x + (W_2 b_1 + b_2).$$

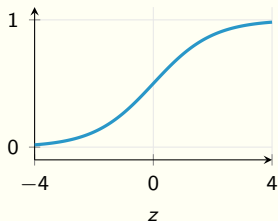
The product $W_2 W_1$ and the bias $W_2 b_1 + b_2$ define one affine map. The same collapse happens at any depth.

A nonlinear activation between the layers stops this collapse. The network can then represent functions whose slope changes.

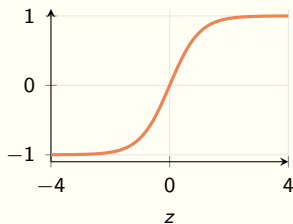
Activation functions

A nonlinear activation bends the affine combination.

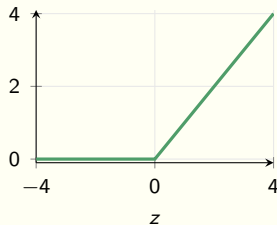
sigmoid



tanh

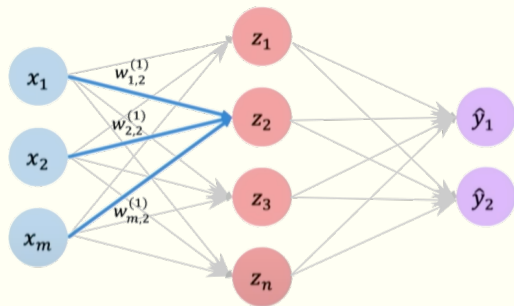


ReLU



ReLU adds a ramp after a breakpoint, while sigmoid and tanh add smooth steps. The output layer stays affine, so the displacement can take either sign.

A network with one hidden layer



Each hidden unit evaluates a shape with its own parameters,

$$h_j(x) = \sigma(w_j x + b_j).$$

The output combines those shapes,

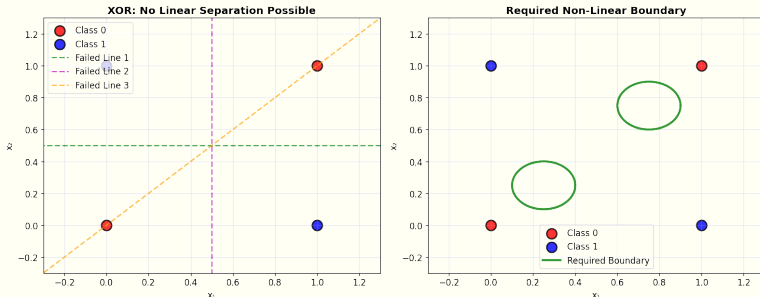
$$\hat{u}(x; \theta) = c + \sum_{j=1}^M v_j h_j(x).$$

With w_j, b_j fixed, fitting v_j and c is linear least squares. Training w_j, b_j as well moves the shapes, and the fit becomes nonlinear.

The XOR problem

XOR assigns the same class to opposite corners of the unit square,

$$(0,0) \mapsto 0, \quad (0,1) \mapsto 1, \quad (1,0) \mapsto 1, \quad (1,1) \mapsto 0.$$



Both diagonals share the midpoint $(1/2, 1/2)$. An affine score that is positive at both class-one corners is positive at that midpoint. If it is also negative at both class-zero corners, it is negative there too, so no line separates the classes.

The best linear classifier for XOR

Predict one when $x_1 + x_2 > 1/2$. The resulting predictions are

(x_1, x_2)	XOR label	linear prediction
(0, 0)	0	0
(0, 1)	1	1
(1, 0)	1	1
(1, 1)	0	1

The best accuracy over affine thresholds is therefore $3/4$. A logistic fit on these four points can settle at probability $1/2$ everywhere and reach only 50 percent, so training can fall short of the best line.

XOR with two ReLU units

Let $s = x_1 + x_2$ for Boolean inputs. The network computes

$$h_1 = \text{ReLU}(s), \quad h_2 = \text{ReLU}(s - 1), \quad \hat{y} = h_1 - 2h_2.$$

s	0	1	2
h_1	0	1	2
h_2	0	0	1
$\hat{y}$	0	1	0

With Boolean inputs, $s = x_1 + x_2$ takes only three values. The input $(0, 0)$ gives $s = 0$, $(0, 1)$ and $(1, 0)$ both give $s = 1$, and $(1, 1)$ gives $s = 2$. Those are the three columns.

$\text{ReLU}(z) = \max(0, z)$ passes positive numbers through unchanged and turns negative numbers into 0, so $h_2 = \text{ReLU}(-1) = 0$ at $s = 0$.

For the column $s = 2$, the input $(1, 1)$ gives $h_1 = \text{ReLU}(2) = 2$, $h_2 = \text{ReLU}(2 - 1) = 1$, and $\hat{y} = 2 - 2(1) = 0$.

The output is linear in the hidden features h_1, h_2 . Those features are nonlinear in the inputs x_1, x_2 .

The XOR features and the decision band

The feature map

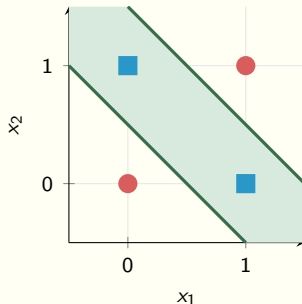
$\phi(x_1, x_2) = (\text{ReLU}(s), \text{ReLU}(s - 1))$ with
 $s = x_1 + x_2$ sends

$$\begin{aligned}(0, 0) &\mapsto (0, 0), & (1, 1) &\mapsto (2, 1), \\ (0, 1), (1, 0) &\mapsto (1, 0).\end{aligned}$$

In feature coordinates an affine threshold separates the labels,

$$\hat{y}_{\text{class}} = \mathbf{1}\{h_1 - 2h_2 > 1/2\}.$$

In the input plane this rule keeps the green band
 $1/2 < x_1 + x_2 < 3/2$, which holds the class-one
corners.



The XOR construction in code

The explicit ReLU parameters can be checked at all four inputs without any training.

```
import numpy as np

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])
s = X.sum(axis=1)
h1 = np.maximum(s, 0)
h2 = np.maximum(s - 1, 0)
prediction = h1 - 2 * h2
assert np.array_equal(prediction, y)
```

Training a network to find such weights is a separate experiment in the companion notebook.

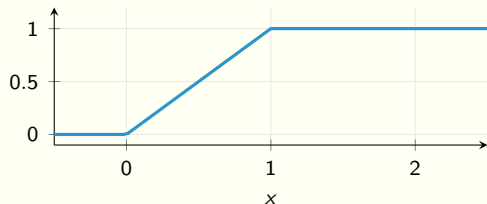
Ramps and triangles from ReLU units

A ReLU ramp is zero before its breakpoint and rises after it. Subtracting a shifted ramp flattens it into a plateau, and a third ramp brings it back to zero,

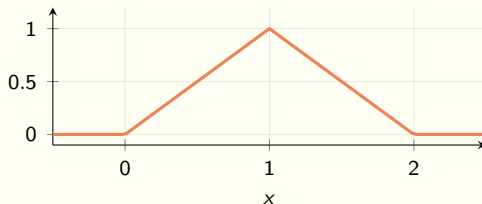
$$r(x) = \text{ReLU}(x) - \text{ReLU}(x - 1),$$

$$t(x) = \text{ReLU}(x) - 2 \text{ReLU}(x - 1) + \text{ReLU}(x - 2).$$

ramp followed by a plateau, $r(x)$



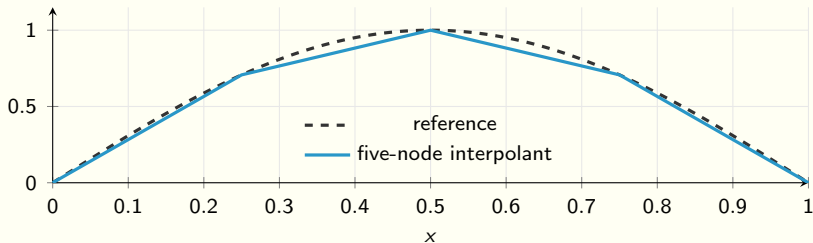
triangle, $t(x)$



A piecewise-linear interpolant as a sum of ReLU terms

A piecewise-linear curve changes slope only at its nodes, and each slope change is one ReLU term,

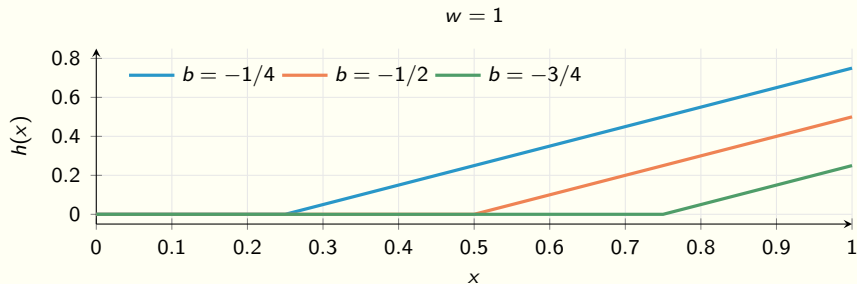
$$p(x) = a + s_0x + \sum_{j=1}^N (s_j - s_{j-1}) \text{ReLU}(x - t_j).$$



Five nodes on the sine give three interior breakpoints, so three ReLU terms and four affine pieces.

The breakpoint of a ReLU unit

For $h(x) = \text{ReLU}(wx + b)$ with $w \neq 0$, the breakpoint sits at $t = -b/w$, and the unit is active where $wx + b > 0$.

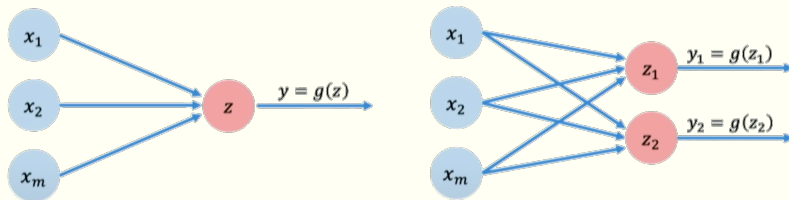


At $w = 1$, changing b slides the breakpoint, and the output weight sets the slope change there. Training moves the breakpoints, with no promise that it finds the best places.

Networks with several outputs

A shared hidden representation can predict several components,

$$\hat{\mathbf{u}}(\mathbf{x}; \theta) = W_{\text{out}} \mathbf{h}(\mathbf{x}) + \mathbf{c}.$$



The output components may be displacements, velocities, or stresses. Predicting a mean and a variance needs a likelihood and a matching loss.

Sup, L^2 , and H^1 errors

To say that a network approximates u , we must say how we measure the gap,

$$\|u - \hat{u}\|_{\infty} = \max_{x \in [0,1]} |u(x) - \hat{u}(x)|, \quad \|u - \hat{u}\|_{L^2}^2 = \int_0^1 |u - \hat{u}|^2 dx.$$

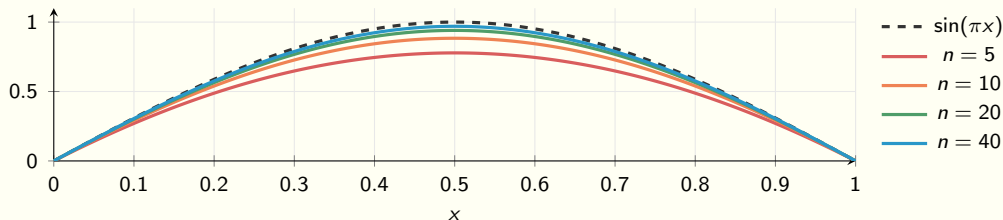
- $C([0,1])$ with the sup norm is a Banach space, a complete normed space.
- $L^2(0,1)$ is a Hilbert space, complete in the norm of its inner product.
- Approximation in $H^1(0,1)$ also requires the weak derivatives to be close in L^2 .

Each density theorem below states its space, its norm, and its assumption on the activation.

The Weierstrass approximation theorem

Weierstrass's theorem

For every continuous f on $[a, b]$ and every $\epsilon > 0$, some polynomial p satisfies $|f(x) - p(x)| < \epsilon$ throughout the interval.



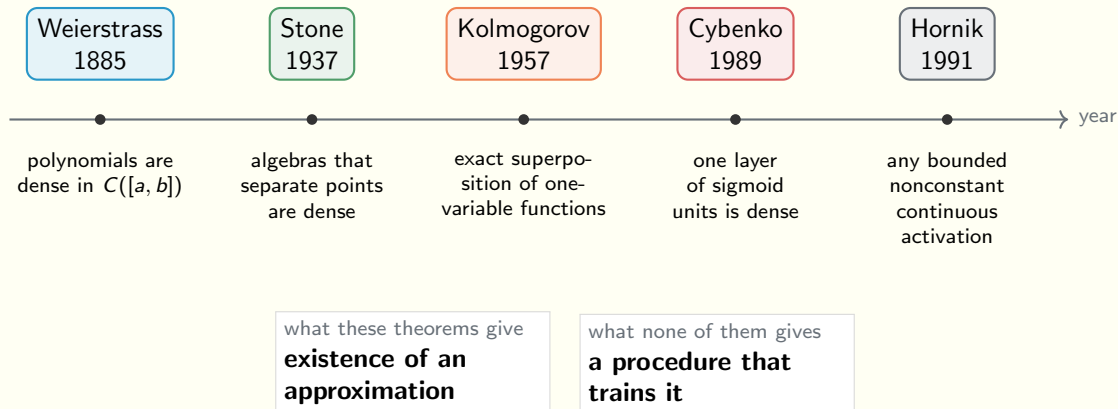
Bernstein polynomials make the theorem concrete for $\sin(\pi x)$. Doubling n roughly halves the largest gap, which is 0.222, 0.117, 0.060, and 0.030 at $n = 5, 10, 20$, and 40. The theorem promises that a polynomial exists, and it names no degree for a given ϵ .

Polynomials and neural networks compared

	polynomials	one-hidden-layer ReLU networks
shapes	fixed global basis	shifted ramps
fitting	linear in basis coefficients	linear in output coefficients when hidden parameters are fixed
locations	prescribed basis	trainable breakpoints
parameters	$\binom{n+d}{d}$ at degree n in d variables	$m(d+2)+1$ for m units and a scalar output
limitations	conditioning and sample placement	optimization and insufficient observations

Neither count says how large a degree or a width a tolerance needs.

History of universal approximation theorems



Cybenko's theorem

Let σ be a continuous sigmoid with limits 0 and 1 at negative and positive infinity. A one-hidden-layer network forms sums

$$G(\mathbf{x}) = \sum_{j=1}^N \alpha_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j).$$

For every $f \in C([0, 1]^d)$ and $\epsilon > 0$, some width and parameters satisfy

$$\max_{\mathbf{x} \in [0, 1]^d} |f(\mathbf{x}) - G(\mathbf{x})| < \epsilon.$$

The width may grow as the tolerance shrinks. The theorem gives no procedure for finding the weights from data. Cybenko (1989).

The proof of Cybenko's theorem

Consider the span S of all shifted and scaled continuous sigmoid units. If its closure were a proper subspace of $C([0, 1])$, Hahn–Banach would supply a nonzero bounded functional with

$$\Lambda(g) = 0 \quad \text{for every } g \in S.$$

The Riesz representation theorem writes that functional as

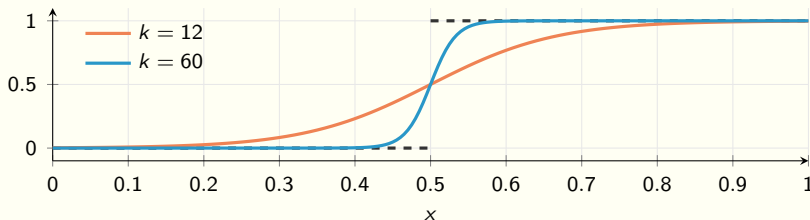
$$\Lambda(g) = \int_0^1 g(x) d\mu(x).$$

Cybenko's sigmoid argument forces the signed measure μ to vanish. This contradicts the choice of a nonzero functional, so the closure of S is all of $C([0, 1])$.

Approximating a step function

Let $H(x) = 0$ for $x < 1/2$, $H(x) = 1$ for $x > 1/2$, and $H(1/2) = 1/2$. Smooth transitions converge pointwise to this function,

$$g_k(x) = \frac{1}{1 + \exp[-k(x - 1/2)]}.$$



The transition narrows, but every continuous function misses H by at least $1/2$ somewhere. The L^2 error of g_k goes to zero.

Leshno's theorem for non-polynomial activations

For a continuous non-polynomial activation, trainable affine arguments and output coefficients give dense one-hidden-layer families in $C([0, 1]^d)$.

- Sigmoid and tanh are smooth and bounded.
- Their derivatives become small in saturated regions.
- ReLU is continuous, non-polynomial, and unbounded.
- Its derivative jumps at zero.

The theorem says nothing about derivatives. The PDE decides how many derivatives the network must approximate.

Leshno et al. (1993), with continuity as a sufficient regularity assumption.

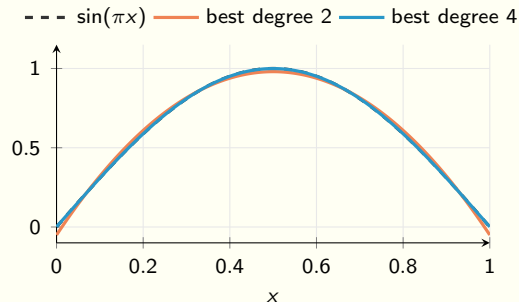
Networks with square activations

With $\sigma(z) = z^2$ and one hidden layer, any width gives

$$\hat{u}(x) = c + \sum_{j=1}^M v_j (w_j x + b_j)^2,$$

a polynomial of degree at most two. More units change the coefficients and never raise the degree.

With L square layers the degree is at most 2^L . Adding layers raises the possible degree.



one square layer, best in L^2
largest gap 0.0503

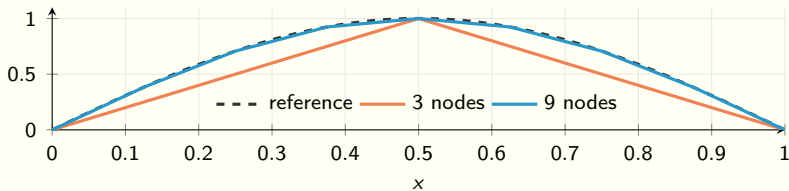
two square layers, best in L^2
largest gap 0.0013

An interpolation error bound

For a twice continuously differentiable u , piecewise-linear interpolation with maximum spacing h satisfies

$$\|u - I_h u\|_\infty \leq \frac{h^2}{8} \max_{x \in [0,1]} |u''(x)|.$$

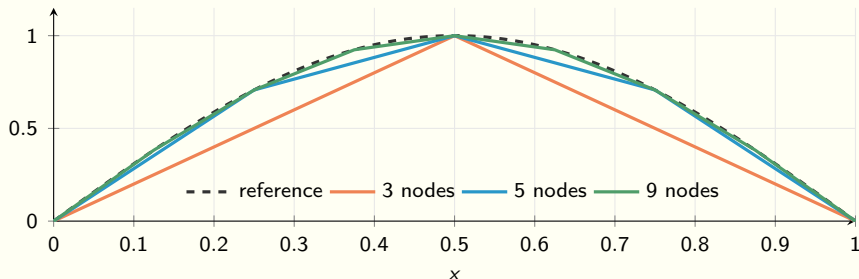
For $u(x) = \sin(\pi x)$, the curvature bound is π^2 .



With 9 nodes, $h = 1/8$ and the bound is $\pi^2/512 = 0.0193$, while the measured largest gap is 0.0188. Halving h divides the bound by four.

Interpolants of the sine with 3, 5, and 9 nodes

Joining closer samples of the sine gives a concrete approximating sequence. Each interpolant is an affine term plus ReLU terms.



The largest gaps are 0.211, 0.070, and 0.019 for 3, 5, and 9 nodes. Here we place the breakpoints by hand. Can training place them?

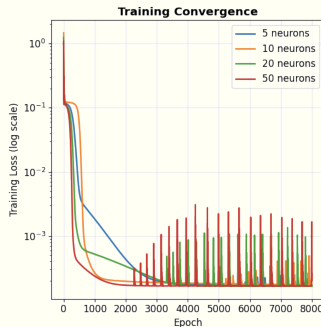
The training loop

For measured displacements, training minimizes the data loss

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{n} \sum_i (\hat{u}(x_i; \theta) - y_i)^2.$$

- 1 Evaluate the network at the readings.
- 2 Compute the residuals and the loss.
- 3 Compute the parameter gradients by AD.
- 4 Update the parameters and check validation error.

A stopping rule ends training, with no guarantee of a global minimum.



Training loss for widths 5 to 50 falls quickly, flattens, and then spikes.

A logistic classifier in code

The sigmoid turns an affine score into a class probability. Binary cross-entropy gives the gradients used in train.

```
import numpy as np

class LogisticClassifier:
    def __init__(self, input_dim):
        self.w = np.zeros(input_dim)
        self.b = 0.0
    def probability(self, X):
        z = X @ self.w + self.b
        return 1 / (1 + np.exp(-z))
    def predict(self, X):
        return self.probability(X) > 0.5
    def train(self, X, y, lr=0.01, epochs=100):
        for _ in range(epochs):
            error = self.probability(X) - y
            self.w -= lr * (X.T @ error) / len(y)
            self.b -= lr * error.mean()
```

Affine regression

For regression, the output is the predicted number itself, $\hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$. The squared-error loss and its gradient are

$$\mathcal{L} = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2,$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij},$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i).$$

Gradient descent moves each parameter against its gradient.

Symbolic, numerical, and automatic differentiation

Symbolic differentiation forms an expression. For $q(x) = x^2 \sin x$, it gives

$$q'(x) = 2x \sin x + x^2 \cos x.$$

Finite differences approximate a derivative,

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

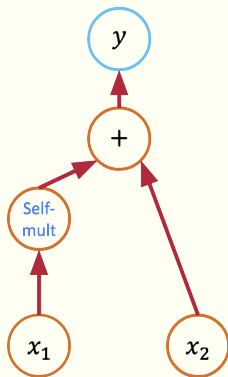
For a smooth function, the forward-difference error includes

$$O(h) + O(\varepsilon_{\text{mach}}/h).$$

Reverse-mode AD computes the gradient of a scalar loss with respect to many parameters in one sweep.

Automatic differentiation applies the chain rule to the executed operations. It computes the program's derivatives up to floating-point error.

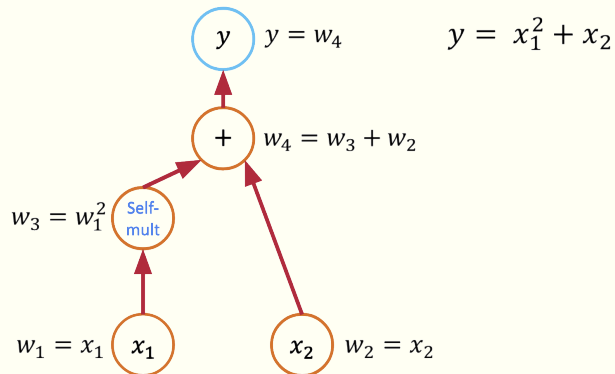
The computational graph for $y = x_1^2 + x_2$



$$y = x_1^2 + x_2$$

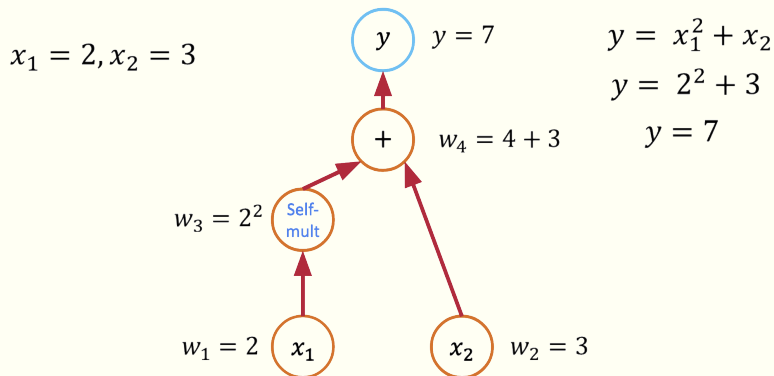
The graph evaluates a square, then adds x_2 .

Name the intermediate values



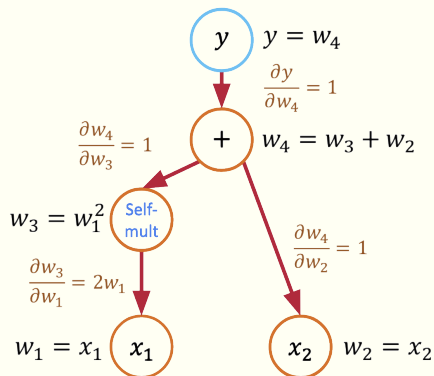
Each w_i names a stored intermediate value.

Evaluate the graph forward



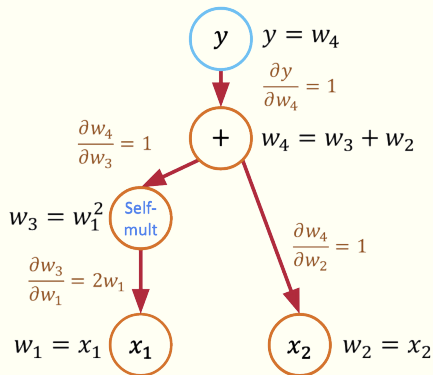
At $x_1 = 2$ and $x_2 = 3$, the square gives 4 and the output is 7.

Differentiate each operation



The addition has local derivatives 1 and 1, while the square has derivative $2w_1$.

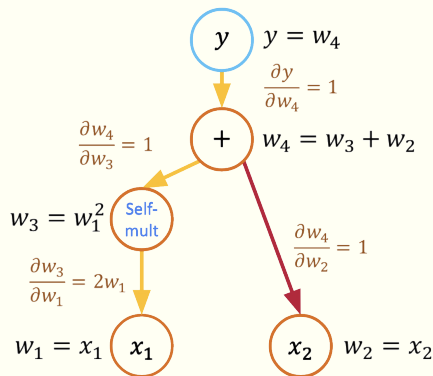
Start the reverse pass at the output



$$\nabla y = \left(\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2} \right)$$

Starting with $\partial y / \partial y = 1$, propagate output sensitivities toward the inputs.

Multiply derivatives along the path to x_1



$$\nabla y = \left(\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2} \right)$$

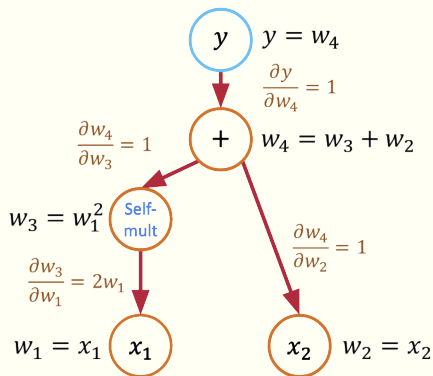
$$\frac{\partial y}{\partial x_1} = \frac{\partial y}{\partial w_1} = \frac{\partial y}{\partial w_4} \frac{\partial w_4}{\partial w_3} \frac{\partial w_3}{\partial w_1}$$

$$\frac{\partial y}{\partial x_1} = 1 * 1 * 2x_1 = 2x_1$$

Original function: $y = x_1^2 + x_2$

The chain rule gives $\partial y / \partial x_1 = 1 \cdot 1 \cdot 2x_1 = 2x_1$.

Collect the derivatives with respect to both inputs



$$y = x_1^2 + x_2$$

$$\nabla y = \left(\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2} \right)$$

$$\nabla y = (2x_1, 1)$$

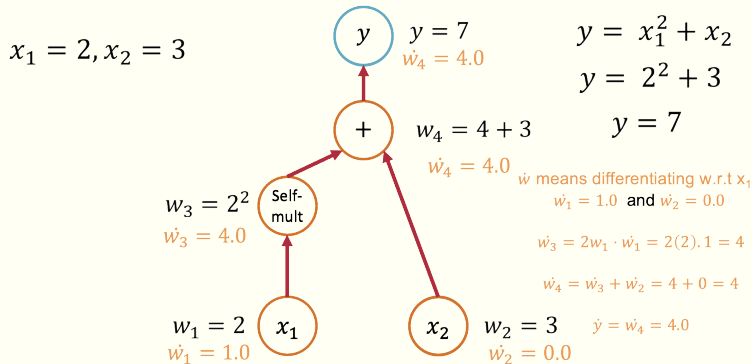
We can find the gradient with respect to the output y

Using $x_1 = 2, x_2 = 3$:

$$\nabla y = (4, 1)$$

The gradient of y with respect to (x_1, x_2) is $(4, 1)$ at $(2, 3)$.

Forward mode on the same graph



Forward mode carries a tangent $\dot{w}_i = \partial w_i / \partial x_1$ with each value, seeded with $\dot{w}_1 = 1$ and $\dot{w}_2 = 0$. One forward sweep gives $\dot{y} = 4 = \partial y / \partial x_1$, and a second sweep seeded at x_2 gives $\partial y / \partial x_2 = 1$. The reverse pass gave both derivatives in one sweep.

Forward mode and reverse mode

For $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$, let J denote its Jacobian. AD computes products with that Jacobian.

Forward mode propagates a chosen input direction $\mathbf{v}$,

$$J\mathbf{v}.$$

Reverse mode propagates a chosen output weighting $\mathbf{a}$,

$$J^T \mathbf{a}.$$

The full Jacobian takes one forward sweep per input direction.

A scalar loss needs one reverse sweep for its whole parameter gradient.

Training has one scalar loss and many parameters, so it uses reverse mode.

Backpropagation

A scalar loss depends on many network parameters. A forward pass stores the intermediate values needed by the chain rule. A reverse pass accumulates their sensitivities to the loss,

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}).$$

For componentwise activations, the hidden-layer sensitivities satisfy

$$\boldsymbol{\delta}^{(l)} = (W^{(l+1)})^T \boldsymbol{\delta}^{(l+1)} \odot \sigma'(\mathbf{z}^{(l)}),$$

where $\odot$ denotes componentwise multiplication.

The forward pass for one reading

For one observation, take $x = 1/2$ and $y = 1$. A one-unit model evaluates

$$z = wx + b, \quad h = \text{ReLU}(z), \quad \hat{y} = vh + c.$$

At $w = v = 1$ and $b = c = 0$, the intermediate values are

$$z = h = \frac{1}{2}, \quad \hat{y} = \frac{1}{2}.$$

The squared-error loss is

$$\ell = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{8}.$$

The backward pass reuses these intermediate values.

The backward pass for one reading

The forward pass gives $\hat{y} - y = -1/2$ and $\text{ReLU}'(z) = 1$. The chain rule gives

$$\frac{\partial \ell}{\partial c} = \hat{y} - y = -\frac{1}{2},$$

$$\frac{\partial \ell}{\partial v} = (\hat{y} - y)h = -\frac{1}{4},$$

$$\frac{\partial \ell}{\partial b} = (\hat{y} - y)v \text{ReLU}'(z) = -\frac{1}{2},$$

$$\frac{\partial \ell}{\partial w} = (\hat{y} - y)v \text{ReLU}'(z)x = -\frac{1}{4}.$$

For step size $\eta = 0.1$, the output weight updates from 1 to 1.025. The larger output weight raises the predicted displacement toward $y = 1$.

Backpropagation in code

The output is linear and the hidden activation is tanh. The code computes gradients of half the mean squared error.

```
import numpy as np

def backpropagation(X, y, W1, b1, W2, b2):
    y = np.asarray(y).reshape(-1, 1)
    n = len(y)
    h = np.tanh(X @ W1 + b1)
    prediction = h @ W2 + b2
    error = prediction - y
    loss = 0.5 * np.mean(error ** 2)
    delta2 = error / n
    dW2 = h.T @ delta2
    db2 = delta2.sum(axis=0)
    delta1 = (delta2 @ W2.T) * (1 - h ** 2)
    dW1 = X.T @ delta1
    db1 = delta1.sum(axis=0)
    return dW1, db1, dW2, db2, loss
```

Full-data and mini-batch gradients

A full-data update differentiates the loss on all observations,

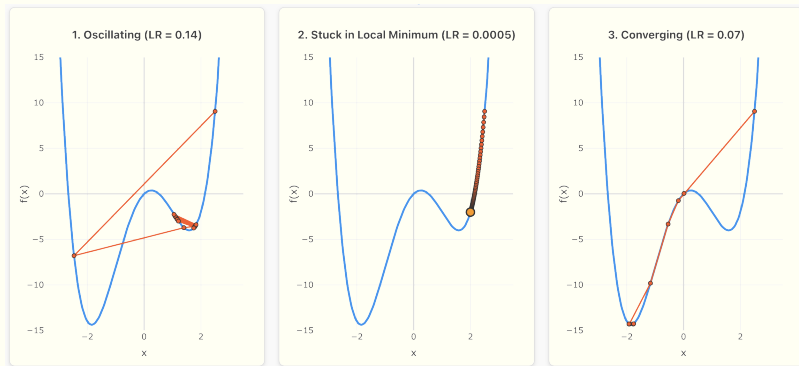
$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{D}}(\theta_t).$$

A mini-batch update uses a subset $\mathcal{B}_t$,

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{B}_t}(\theta_t).$$

Uniform random batches give unbiased estimates of the full-data gradient. Their sampling noise depends on the batch size. Fifteen field readings fit in one full-data batch.

The step size in gradient descent



Here is gradient descent $x_{t+1} = x_t - \eta f'(x_t)$ on a function with two minima, from the same start. A step of $\eta = 0.14$ overshoots and oscillates, and a step of $\eta = 0.0005$ creeps into the nearer minimum. A step of $\eta = 0.07$ reaches the deeper minimum.

Momentum and Adam

Momentum averages recent gradients $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$,

$$v_t = \beta v_{t-1} + (1 - \beta)g_t, \quad \theta_{t+1} = \theta_t - \eta v_t.$$

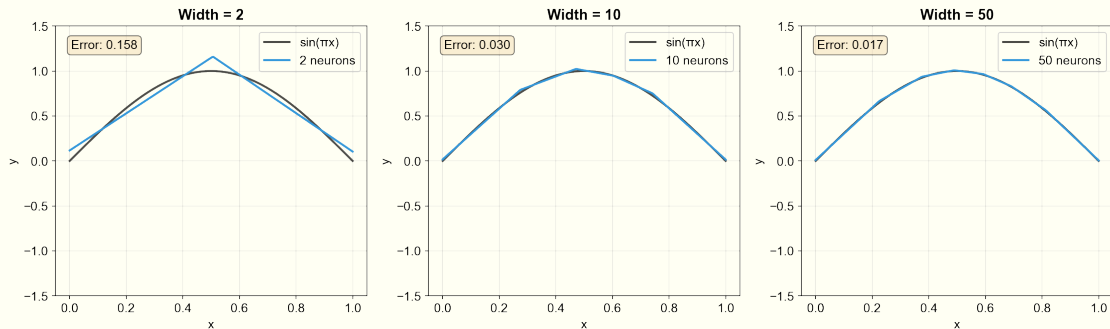
Adam also averages squared gradients componentwise,

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1)g_t, \\ s_t &= \beta_2 s_{t-1} + (1 - \beta_2)g_t^2, \\ \hat{m}_t &= m_t / (1 - \beta_1^t), \quad \hat{s}_t = s_t / (1 - \beta_2^t), \\ \theta_{t+1} &= \theta_t - \eta \hat{m}_t / (\sqrt{\hat{s}_t} + \epsilon). \end{aligned}$$

The averages start at zero, and the hats correct that bias.

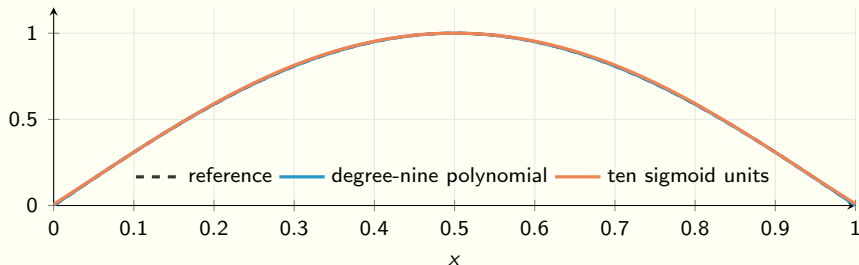
Trained ReLU networks of width 2, 10, and 50

Universal Approximation: Effect of Network Width



Now the network places its own breakpoints. Each panel is the best of five Adam runs on 100 samples of $\sin(\pi x)$. The largest gap on 200 points falls from 0.158 at width 2 to 0.030 at width 10 and 0.017 at width 50.

Polynomial and neural network approximation of $\sin(\pi x)$

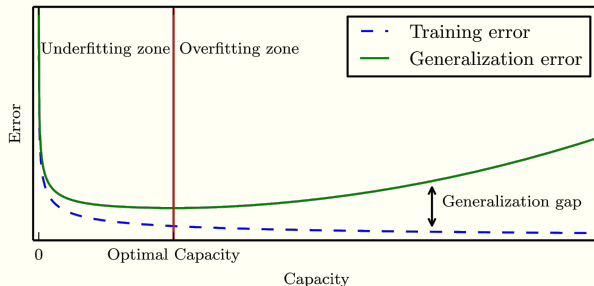


polynomial, ten coefficients
largest sampled gap
 $8.23e - 08$

network, thirty-one
parameters
largest sampled gap
0.00964

Both fit the same 100 clean samples. For one smooth function of one variable, the polynomial beats this trained network by five orders of magnitude.

Training error and validation error



A falling training loss can come with a rising validation loss.

- Keep the validation readings out of the fit.
- Choose validation points that match the intended prediction task.
- Check derivatives separately when the task uses derivatives.

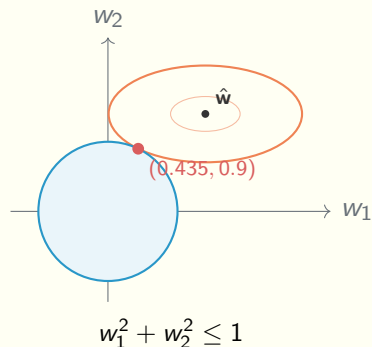
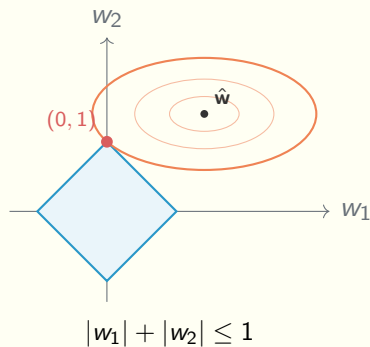
Sparse observations leave many functions with similar data loss. A penalty adds a preference among their parameters,

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda \mathcal{R}(\theta).$$

- L1 penalties can favor sparse coefficients.
- L2 penalties discourage large parameter magnitudes.
- Early stopping also changes the fitted result.

Validation decides whether a choice improves prediction. Small weights alone do not guarantee a smooth or accurate field.

L1 and L2 constraint sets



Both panels use the loss $(w_1 - 1.4)^2 + 4(w_2 - 1.4)^2$, whose contours are ellipses around $\hat{\mathbf{w}} = (1.4, 1.4)$. The constrained minimizer is where the smallest contour first meets the constraint set. The diamond is met at its corner $(0, 1)$, so $w_1 = 0$, while the circle is met where both coefficients are nonzero.

The L1 penalty

The L1-penalized objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_i |w_i|.$$

For $w_i \neq 0$, the penalty derivative is $\lambda \text{sign}(w_i)$. At zero, its subdifferential is $[-\lambda, \lambda]$.

A proximal step uses soft thresholding,

$$w_i^{\text{new}} = \text{sign}(z_i) \max(|z_i| - \eta\lambda, 0),$$

where z_i is the result of a data-loss gradient step. Ordinary gradient steps on $|w_i|$ rarely land on zero.

The L2 penalty

The squared L2 penalty gives

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_i w_i^2, \quad \frac{\partial(\lambda \mathcal{R})}{\partial w_i} = 2\lambda w_i.$$

A gradient step on the penalty alone shrinks every coefficient by the same factor,

$$w_i^{\text{new}} = (1 - 2\eta\lambda)w_i.$$

For linear least squares with every coefficient penalized and $\lambda > 0$, the penalized coefficients are unique. For a network the fit stays nonconvex, and permuting hidden units leaves both the prediction and the penalty unchanged.

L1 and L2 penalties compared

property	L1 penalty	squared L2 penalty
penalized quantity	$\sum_i w_i $	$\sum_i w_i^2$
regularity	nondifferentiable at zero	differentiable
linear coefficient fit	may have sparse optima	unique if all coefficients are penalized and $\lambda > 0$
neural parameter fit	generally nonconvex	generally nonconvex
response outliers	influence depends on data loss	influence depends on data loss

A penalty acts on the coefficients. Robustness to outlying readings comes from the data loss, for example an absolute-value loss on the residuals.

A weight penalty in code

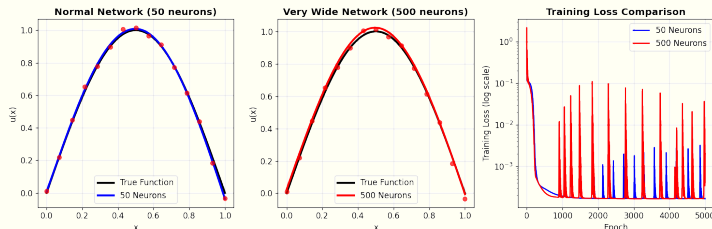
This example penalizes the weight matrices and leaves the biases unpenalized.

```
import torch
from torch import nn

class RegularizedNet(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim,
                  reg_type="l2", lambda_reg=0.01):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)
        self.reg_type, self.lambda_reg = reg_type, lambda_reg
    def forward(self, X):
        return self.fc2(torch.tanh(self.fc1(X)))
    def regularization_loss(self):
        weights = (self.fc1.weight, self.fc2.weight)
        if self.reg_type == "l1":
            penalty = sum(w.abs().sum() for w in weights)
        else:
            penalty = sum(w.square().sum() for w in weights)
        return self.lambda_reg * penalty
```

Choosing the penalty parameter λ

The penalty parameter balances agreement with the readings against the parameter preference.



- Fit candidate values of λ on the same training readings.
- Compare their validation errors in a stated metric.
- Keep separate test readings for the final assessment.

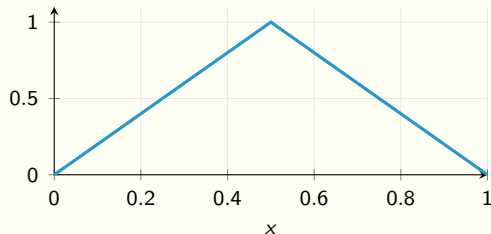
Composing a triangle with itself

Consider the triangle

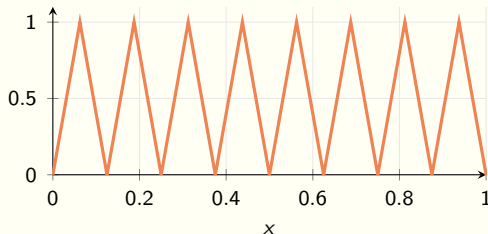
$$T(x) = 2 \operatorname{ReLU}(x) - 4 \operatorname{ReLU}(x - \tfrac{1}{2}) + 2 \operatorname{ReLU}(x - 1).$$

Composing T with itself repeats this shape on smaller intervals.

$T(x)$



$T \circ T \circ T \circ T(x)$



Each composition splits every affine piece in two, so four copies of T give $2^4 = 16$ pieces on $[0, 1]$.

Breakpoint counts and parity

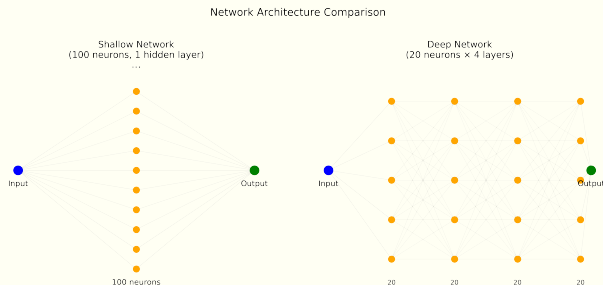
For $T^{\circ k}$, the number of interior breakpoints is $2^k - 1$. A one-hidden-layer scalar-input ReLU sum needs at least that many units, one per breakpoint. The composition uses three units per copy of T , so $3k$ in all.

Parity is a different story. On Boolean inputs, let $s = \sum_{j=1}^n x_j$. A shallow ReLU network with n units represents parity as

$$q(s) = \text{ReLU}(s) + 2 \sum_{j=1}^{n-1} (-1)^j \text{ReLU}(s - j).$$

With the sum s as its input, parity gives no separation between shallow and deep networks.

Parameter counts for a shallow and a deep network



For scalar input and output, the parameter counts are

hidden architecture	weights	biases	total
100	200	101	301
20, 20, 20, 20	1240	81	1321

The deep network has more than four times the parameters of the shallow one. A fair test of depth fixes the parameter count, the data, and the training.

Value error and slope error

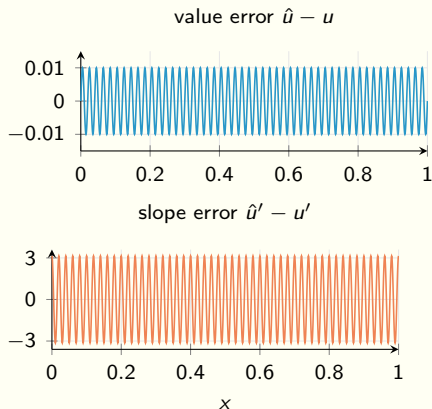
Add a small fast oscillation to any reference u ,

$$\hat{u}(x) = u(x) + 0.01 \sin(100\pi x).$$

The value and slope errors are

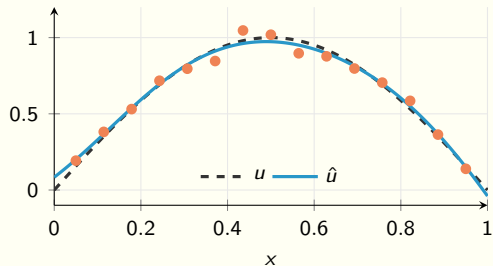
$$\begin{aligned}\|\hat{u} - u\|_{\infty} &= 0.01, \\ \left\| \frac{d\hat{u}}{dx} - \frac{du}{dx} \right\|_{L^2} &= \frac{\pi}{\sqrt{2}} \approx 2.22.\end{aligned}$$

The H^1 norm adds the slope error to the value error. AD computes $d\hat{u}/dx$ to rounding error, and $d\hat{u}/dx$ can still be far from du/dx .



A network fitted to the readings alone

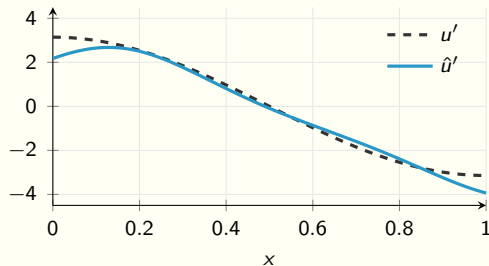
values



RMSE in u
0.0224

RMSE in u'
0.304

slopes



RMS of the residual
 $-\hat{u}'' - f$
3.27

A tanh network with 12 units, fitted to the fifteen readings alone, matches the values far better than the slopes or the equation. How can the known equation enter the fit?

The data loss and the residual loss

For displacement observations, squared error measures value mismatch. For the known Poisson equation, the residual measures equation mismatch,

$$\mathcal{L}_{\text{data}} = \frac{1}{n} \sum_i (\hat{u}(x_i; \theta) - y_i)^2,$$
$$r(x; \theta) = -\frac{d^2 \hat{u}}{dx^2}(x; \theta) - f(x),$$
$$\mathcal{L}_{\text{PDE}} = \frac{1}{m} \sum_{k=1}^m r(z_k; \theta)^2.$$

The residual points z_k need not coincide with the observation positions x_i . The boundary values need terms of their own.

The PINN loss for the string

For the pinned string, define the Poisson residual

$$r(x; \theta) = -\frac{d^2 \hat{u}}{dx^2}(x; \theta) - \pi^2 \sin(\pi x).$$

A sampled objective can combine residuals, boundaries, and observations,

$$\mathcal{L}(\theta) = \frac{\lambda_r}{m} \sum_{k=1}^m r(z_k; \theta)^2 + \lambda_b (\hat{u}(0; \theta)^2 + \hat{u}(1; \theta)^2) + \frac{\lambda_d}{n} \sum_{i=1}^n (\hat{u}(x_i; \theta) - y_i)^2.$$

AD differentiates $\hat{u}$ with respect to x for the residual and $\mathcal{L}$ with respect to θ for training. A ReLU network has $d^2 \hat{u}/dx^2 = 0$ away from its breakpoints, so its residual there equals $-\pi^2 \sin(\pi x)$. Smooth activations such as tanh supply the second derivative the residual needs.

Three losses for one field

The same coordinate representation can be used with different information,

$$\hat{u}(x; \theta).$$

displacement
observations

$$\mathcal{L}_{\text{data}}$$

known equation
and boundaries

$$\mathcal{L}_{\text{PDE}} + \mathcal{L}_{\text{BC}}$$

physics and
observations
combined objective

The network is the same in all three, and only the loss changes.

A checklist for fitting a field

- Scale coordinates and measured quantities consistently.
- Choose activation regularity for the required derivatives.
- Check gradients on a small worked problem.
- Record seeds and optimization settings.
- Monitor training and validation errors.
- Evaluate PDE residuals on points outside the training set.
- Check boundary values and derivative errors.
- Compare against a simpler representation.

Batch normalization makes one coordinate's prediction depend on the other points in the batch, which corrupts coordinate derivatives.

Summary of the approximation results

Approximation theorems concern the represented family, for a stated target class and error measure.

- Polynomials and suitable neural families approximate continuous targets uniformly.
- No continuous network approximates a jump uniformly to arbitrary accuracy.
- Derivative approximation needs further assumptions on the activation and the function space.

Finitely many readings do not identify a continuous target. Training and validation answer questions that density does not.

Summary of training and architecture

Training chooses parameters within a representation, using the gradients of the chosen objective.

- Reverse-mode AD computes a scalar loss's parameter gradient.
- Regularization changes which fits the objective prefers.
- Depth can reduce representation size for specified compositional targets.
- Validation checks the fitted result on the intended prediction task.

Neither density nor a gradient computation guarantees that training recovers the field.

The network represents a displacement as a parameterized function. Hidden parameters change its shapes, while output coefficients combine them.

- ReLU terms construct changing slopes and piecewise-linear approximations.
- A loss specifies agreement with data, physics, or both.
- The chain rule supplies parameter gradients for training.
- Approximation claims require a function class, an architecture, and a norm.
- Values, derivatives, and residuals require separate checks.

For the Poisson equation, a smooth coordinate network supplies the derivatives that the strong residual needs.

Questions?

Thank you!

krishnak@utexas.edu

<https://github.com/kks32-courses/sciml>